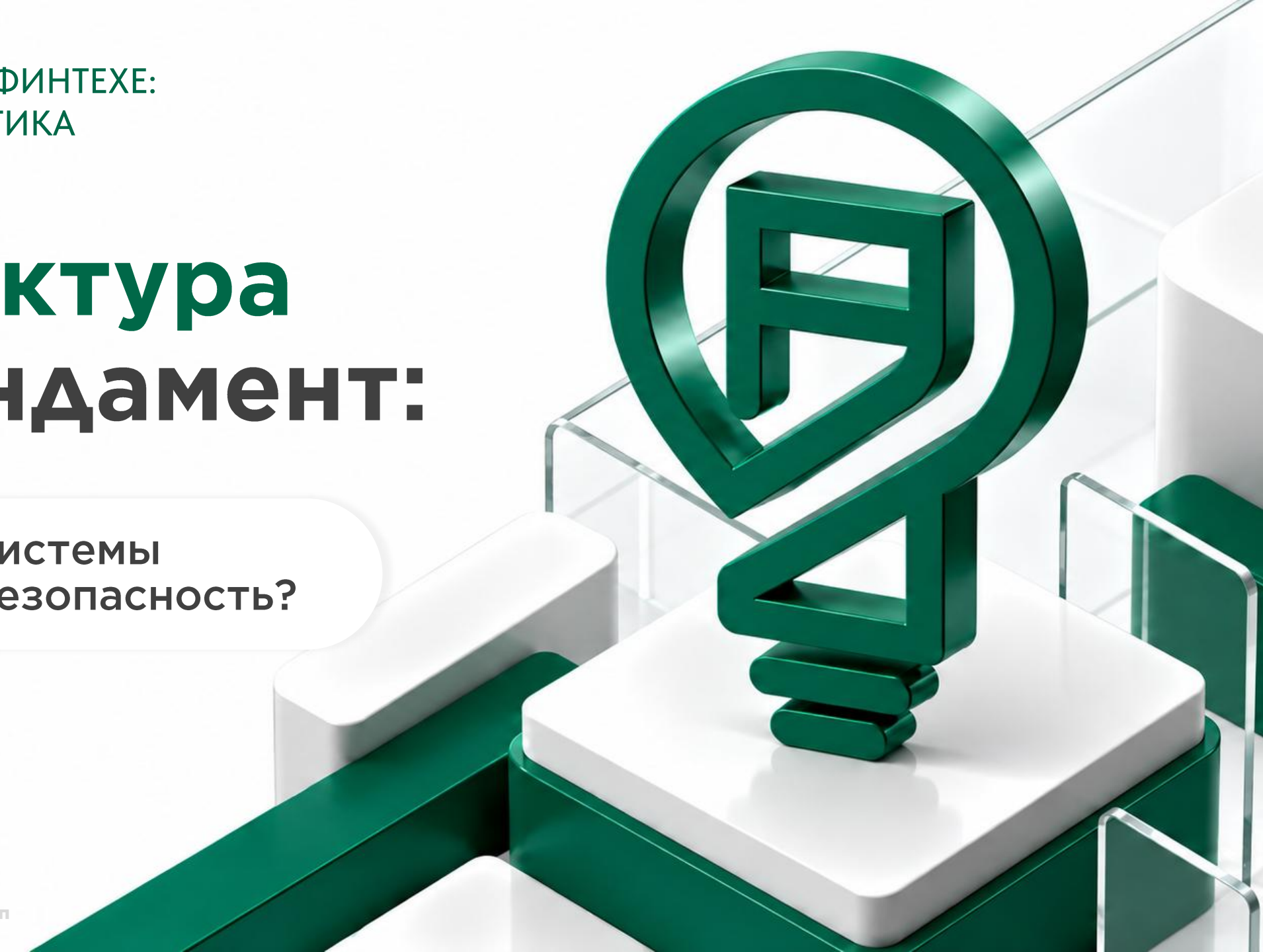
ИННОВАЦИИ В ФИНТЕХЕ:
ТЕОРИЯ И ПРАКТИКА

Архитектура как фундамент:

почему дизайн системы определяет её безопасность?

Стоделов
Денис



НСПК | МИР | СБП

раздел

01

Введение



НСПК | МИР | СБП

Архитектура как фундамент

Безопасность нельзя добавить в конце, как украшение

Она должна быть встроена в архитектуру с самого начала. Архитектура определяет границы доверия, потоки данных, точки отказа

Три базовых принципа:

01 эшелонированная защита

02 Zero Trust

03 триада CIA



Принцип эшелонированной защиты (Defense in Depth)

01 Многоуровневая защита: сеть, хост, приложение, данные

02 Противник должен преодолеть несколько разнородных барьеров

03 Примеры уровней: WAF, EDR, шифрование, MFA

04 Повышает стоимость атаки и даёт время на реагирование

Принцип нулевого доверия (Zero Trust)

01 «Никому не доверяй, всегда проверяй»

02 Полный отказ от концепции доверенной внутренней сети

03 Непрерывная аутентификация и авторизация каждого запроса

04 Микросегментация и соблюдение принципа наименьших привилегий (least privilege)

Конфиденциальность, целостность, доступность (CIA)



01 **Конфиденциальность** — данные доступны только тем, кому положено

02 **Целостность** — данные не изменены несанкционированно

03 **Доступность** — данные и сервисы доступны тогда, когда это необходимо

Задача архитектора — найти идеальный баланс между этими аспектами

раздел

02

Примеры утечек



НСПК | МИР | СБП

Пример 1: Утечка в Capital One (2019)

Масштаб
Утечка данных 100 млн клиентов

Вектор
SSRF-атака через неправильно настроенный WAF

Архитектурные просчёты

- избыточные права роли IAM
- отсутствие сегментации между публичным приложением и S3-бакетами
- слабый мониторинг аномалий



Пример 2: Взлом Uber (2022)



Масштаб
Полный взлом внутренней ИТ-инфраструктуры

Вектор
Социальная инженерия (MFA fatigue) + хардкод учётных данных

Архитектурные просчёты

- плохое управление секретами (хранение в открытом репозитории)
- отсутствие Zero Trust
- избыточные права доступа

Пример 3: Атака на MoveIT (2023)

Масштаб
Массовая утечка данных тысяч организаций

Вектор
Уязвимость нулевого дня (SQL-инъекция)
в MOVEit Transfer

Архитектурные просчёты

- размещение в одном сегменте с критичными системами
- избыточные права на доступ к файловым хранилищам
- отсутствие эшелонированной защиты



Пример 4: Change Healthcare (2024)



Масштаб
Крупнейшая утечка медицинских данных в США

Вектор
Отсутствие MFA на удалённом портале Citrix

Архитектурные просчёты

- слабая аутентификация на периметре
- плоская внутренняя сеть без сегментации
- избыточные привилегии сервисных учёток

Общие выводы

Большинство утечек это:



не результат одной ошибки



а сочетания архитектурных просчётов

Повторяющиеся из года в год проблемы

- отсутствие MFA
- избыточные права
- плоская сеть
- хардкод секретов

Вывод: безопасность должна быть неотъемлемой частью дизайна, а не патчем в конце

раздел

03

Лучшие практики и требования



НСПК | МИР | СБП

Обзор ключевых областей защиты

01 Сетевое размещение

02 Авторизация

03 Защита конечных точек

04 Передача данных

05 Регистрация событий

06 Безопасность секретов

07 Безопасность интеграций

08 Безопасность API

09 Другие требования

Сетевое размещение компонентов: Требования

01 Сегментация сети: строгое разделение на зоны доверия (DMZ, внутренняя сеть, БД)

02 Микросегментация для изоляции критичных микросервисов

03 Полная изоляция сред: production, staging, development

04 Контроль как входящего, так и исходящего трафика (Firewall, WAF, IDS/IPS)

05 Строгий запрет прямого доступа из интернета к внутренним ресурсам

Сетевое размещение компонентов: Паттерны и Антипаттерны

рекомендуемые паттерны

- DMZ с балансировщиком, WAF и обратным прокси
- Микросегментация на основе identity (service mesh)

опасные антипаттерны

- Плоская сеть, где все видят всех
- Размещение БД в публичной подсети

Пример

В Capital One отсутствие сегментации позволило после SSRF скачать данные из S3

Авторизация пользователей и компонентов: Требования

01 Принцип наименьших привилегий (least privilege) на всех уровнях

02 Гранулярное разграничение доступа: на уровне функций, данных и конкретных полей

03 Централизованная проверка авторизации строго на доверенном сервере

04 Категорический запрет клиентской авторизации (не доверять коду JavaScript)

05 Немедленное применение изменений прав и механизмы отзыва токенов

Авторизация: Паттерны и Антипаттерны

рекомендуемые паттерны

- RBAC/ABAC с централизованным Policy Decision Point (PDP)
- Проверка прав на каждом запросе (Zero Trust)

опасные антипаттерны

- Использование одной учётной записи admin для всех сервисов
- Проверка прав только на стороне клиента

Пример

В Uber после компрометации аккаунта подрядчика злоумышленник получил широкие права из-за отсутствия сегментации прав

Защита конечных точек: Требования

01 Непрерывное управление уязвимостями и обновлениями (patch management)

02 Обязательное использование современных EDR/антивирусных решений

03 Полный запрет небезопасных и устаревших технологий (Flash, ActiveX)

04 Строгий контроль устанавливаемого ПО (механизм allowlist)

05 Обязательное шифрование дисков и защита от эксплойтов (ASLR, DEP)

06 Минимизация состава установленного ПО на серверах

Защита конечных точек: Паттерны и Антипаттерны

рекомендуемые паттерны

- Централизованное управление патчами, автообновление
- Изоляция критичных серверов через jump host или bastion

опасные антипаттерны

- Использование неподдерживаемых операционных систем
- Отключённый автоматический апдейт систем

Пример

Эпидемия WannaCry наглядно показала, как миллионы необновлённых систем Windows стали жертвами сетевого червя

Требования к передаче данных

01 Использование исключительно современных версий TLS (1.2 и 1.3)

02 Шифрование данных в транзите везде, включая внутренние сервисы

03 Обязательная проверка сертификатов, запрет самоподписанных в продакшене

04 Внедрение mTLS для взаимной аутентификации межсервисного взаимодействия

05 Строгий запрет передачи конфиденциальных данных в URL-параметрах

06 Надежная защита от атак типа «человек посередине» (MITM) через HSTS

Передача данных:
Паттерны и Антипаттерны

рекомендуемые паттерны

- TLS termination на балансировщике с шифрованием до бэкенда
- Использование service mesh с автоматическим mTLS

опасные антипаттерны

- Использование открытого HTTP внутри доверенной сети
- Отключение проверки сертификатов на клиентах

Пример

Перехват трафика и утечки данных в мобильных приложениях часто происходят из-за отключенной проверки SSL-сертификатов

Регистрация событий в системе: Требования

01 Логирование всех событий безопасности: аутентификация, авторизация, изменения конфигураций

02 Централизованный сбор логов в единую SIEM-систему

03 Защита логов от изменения, удаления и несанкционированного доступа

04 Обязательная синхронизация времени на всех узлах по протоколу NTP

05 Категорический запрет логирования секретов (паролей, токенов, номеров карт)

06 Постоянный мониторинг и оперативное оповещение об аномалиях

Регистрация событий: Паттерны и Антипаттерны

рекомендуемые паттерны

- Структурированное логирование (JSON) и сбор через fluentd/rsyslog
- Immutable logs (использование WORM-хранилищ)

опасные антипаттерны

- Хранение логов исключительно на локальном диске
- Запись конфиденциальных данных и паролей в логи

Пример

Без централизованного мониторинга и SIEM множество сложных целевых атак остаются незамеченными внутри сети месяцами

Обеспечение безопасности секретов

01 Категорический запрет на хранение секретов в коде, репозиториях и конфигурациях

02 Использование специализированных хранилищ (Vault, AWS Secrets Manager, Key Vault)

03 Регулярная автоматическая или принудительная ротация секретов

04 Строгий принцип наименьших привилегий для доступа приложений к хранилищу

05 Детальный аудит каждого случая доступа к секретам

06 Использование аппаратных модулей безопасности (HSM) для критических задач

Безопасность секретов: Паттерны и Антипаттерны

рекомендуемые паттерны

- Динамические секреты и краткосрочные токены
- Интеграция с CI/CD для инъекции секретов в рантайме

опасные антипаттерны

- Хранение паролей в переменных окружения в открытом виде
- Использование одного мастер-секрета для всех сред (dev/prod)

Пример

В Uber учётные данные подрядчика были найдены хакером в открытом репозитории GitHub, что дало первичный доступ к сети

Безопасность интеграций систем: Требования

01 Обязательная взаимная аутентификация и авторизация всех интеграционных каналов

02 Использование индивидуальных сервисных учётных записей для каждой интеграции

03 Обязательное шифрование каналов связи (TLS, mTLS)

04 Ограничение исходящих соединений инфраструктуры с помощью allowlist

05 Контроль максимального количества одновременных соединений и таймаутов

06 Непрерывный мониторинг аномальной активности на стыках систем

Безопасность интеграций: Паттерны и Антипаттерны

рекомендуемые паттерны

- Применение service mesh с mTLS и строгими политиками трафика
- Использование API Gateway для оркестрации внешних интеграций

опасные антипаттерны

- Прямое подключение к базе данных из внешних систем
- Использование HTTP без шифрования для внутренних вызовов

Пример

В MOVEit отсутствие сегментации и избыточные права интеграционных компонентов привели к компрометации всей внутренней файловой базы

Требования по безопасности API

01 Обязательная аутентификация каждого запроса (протоколы OAuth 2.0, JWT, OpenID Connect)

02 Авторизация на уровне методов и на уровне конкретных объектов данных

03 Строгая валидация всех входных данных по схемам и через allowlist

04 Защита от инъекций всех типов (SQL, NoSQL, команд ОС)

05 Ограничение частоты запросов (rate limiting) для защиты от брутфорса и DoS

06 Запрет на передачу избыточных данных в ответах API (data minimization)

07 Полное логирование и мониторинг вызовов API

Безопасность API:
Паттерны и Антипаттерны

рекомендуемые паттерны

- Размещение API Gateway с функциями rate limiting и валидации
- Короткоживущие access-токены (JWT) + refresh-токены

опасные антипаттерны

- Передача секретных API-ключей в URL-параметрах запроса
- Отсутствие серверной валидации входных данных

Пример

Историческая утечка данных Facebook API произошла из-за отсутствия должного контроля доступа на уровне объектов данных

Другие требования по кибербезопасности

01 Управление конфигурациями (hardening, отключение отладки, скрытие версий ПО)

02 Обработка ошибок без раскрытия внутренней технической информации (стек вызовов, пути)

03 Защита от автоматизированных атак (CAPTCHA, интеллектуальный rate limiting)

04 Управление зависимостями (ведение SBOM, регулярная проверка библиотек)

05 Внедрение практик безопасной разработки (code review, SAST, DAST)

06 Защита от подделки запросов (CSRF, SSRF) и специфика WebRTC, GraphQL, WebSocket

Другие требования:
Паттерны и Антипаттерны

рекомендуемые паттерны

- Инфраструктура как код (IaC) с автоматической проверкой безопасности
- Регулярное сканирование уязвимостей и проведение пентестов

опасные антипаттерны

- Включённый debug-режим в production-окружении
- Использование устаревших и неподдерживаемых библиотек

Пример

Уязвимость Log4Shell (2021) в популярной библиотеке логирования показала критичность контроля сторонних зависимостей

раздел 04

Заключение



НСПК | МИР | СБП

Выводы

- Архитектура — это фундамент безопасности информационной системы
- Базовые столпы: эшелонированная защита, философия Zero Trust и триада CIA
- Громкие инциденты — это почти всегда кумулятивный эффект, а не одна ошибка
- Ключевые практики: сегментация, наименьшие привилегии, шифрование транзита, защита секретов
- Главное правило: Безопасность должна быть встроена с самого начала!

ИННОВАЦИИ В ФИНТЕХЕ:
ТЕОРИЯ И ПРАКТИКА

Спасибо
за внимание!

Стоделов
Денис

НСПК | МИР | СБП

